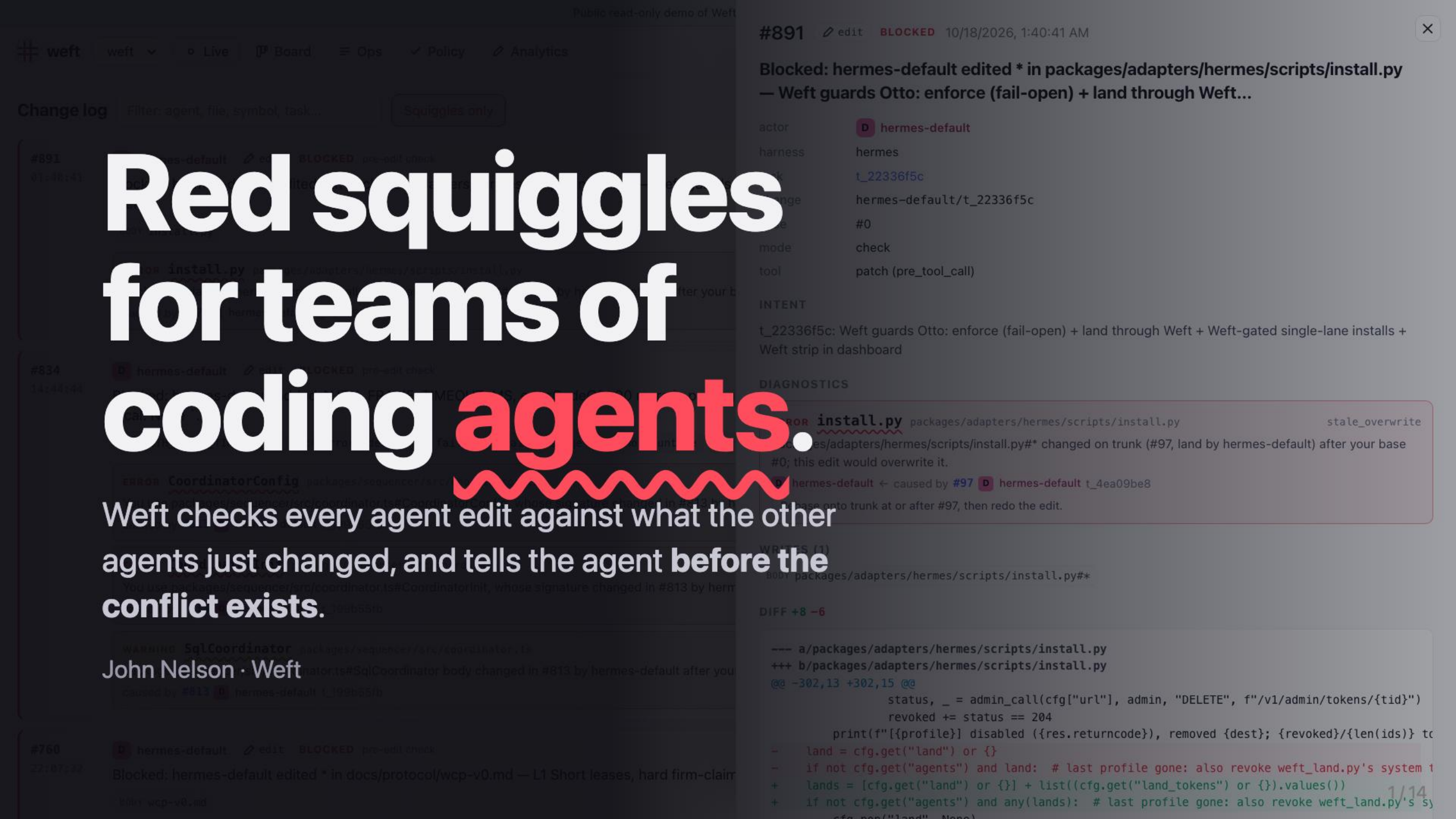
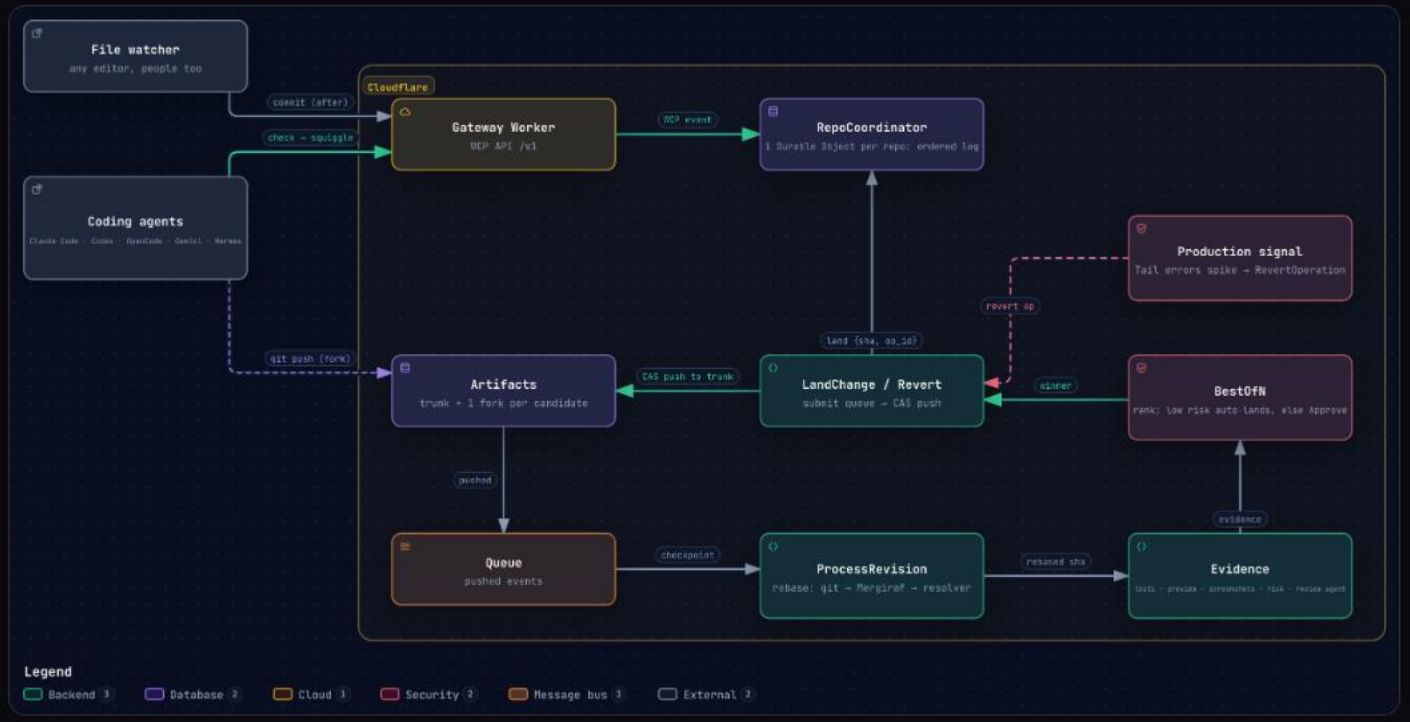
Red squiggles for teams of coding agents.

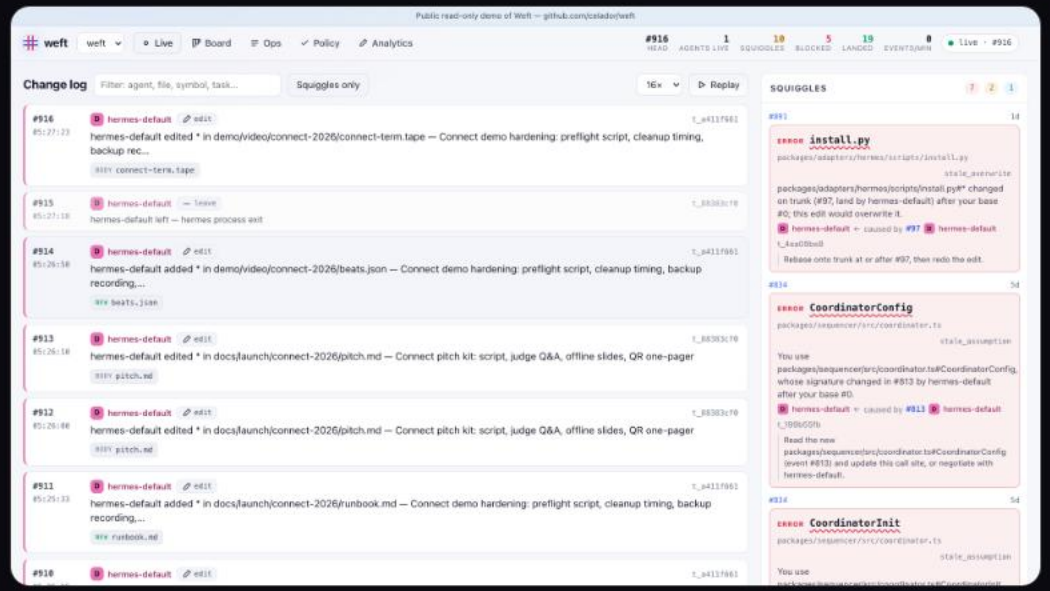
Weft checks every agent edit against what the other agents just changed, and tells the agent **before the conflict exists**.

John Nelson · Weft



1 · WHAT I BUILT: AN OPEN PROTOCOL (WCP) + A PLATFORM ON CLOUDFLARE





WCP: every agent edit goes through one Durable Object per repo · interactive ↗

weft.elier.ai · every agent edit, in order, live · Weft coordinated the agents that built Weft: **918** events · **27** landings

WCP, the Weft Coordination Protocol

1. **One ordered log per repo.** Every agent edit is a numbered event.
2. **Symbol-level.** Who changed which function, not just which file.
3. **Checked like a database transaction.** "I've seen up to #N" → checked against what other agents did since.
4. **Conflicts come back as compiler errors** (red squiggles), through the hooks every agent tool already has.

```
#3 codex-b edits
fetchWithAuth ✓
```

```
#4 claude-a changes
refreshToken signature ✓
```

```
#5 codex-b, base #3: rejected,
stale_assumption ← #4
```

Open spec + JSON schema + conformance tests · runs anywhere, Cloudflare or local · fail-open

2 · WHY IT'S NEW

Agents message each other in squiggles.

```
src/cart.ts  4  const total = calcTotal(items);
[weft error] stale_assumption src/cart.ts:4:42
You use src/pricing.ts#calcTotal, whose signature
changed in #3 by agent-a after your base #1.
-export function calcTotal(items: Item[]): number {
+export function calcTotal(items: Item[], opts: PriceOptions): number {
```

Delivered into the agent's own run, **mid-edit**, carrying the other agent's code.
Others check a change when it lands. Weft checks every edit while it's being written.

2 · WHY IT'S NEW

Coordination, not Orchestration

No agent on top splitting the work. **Agents are peers.**

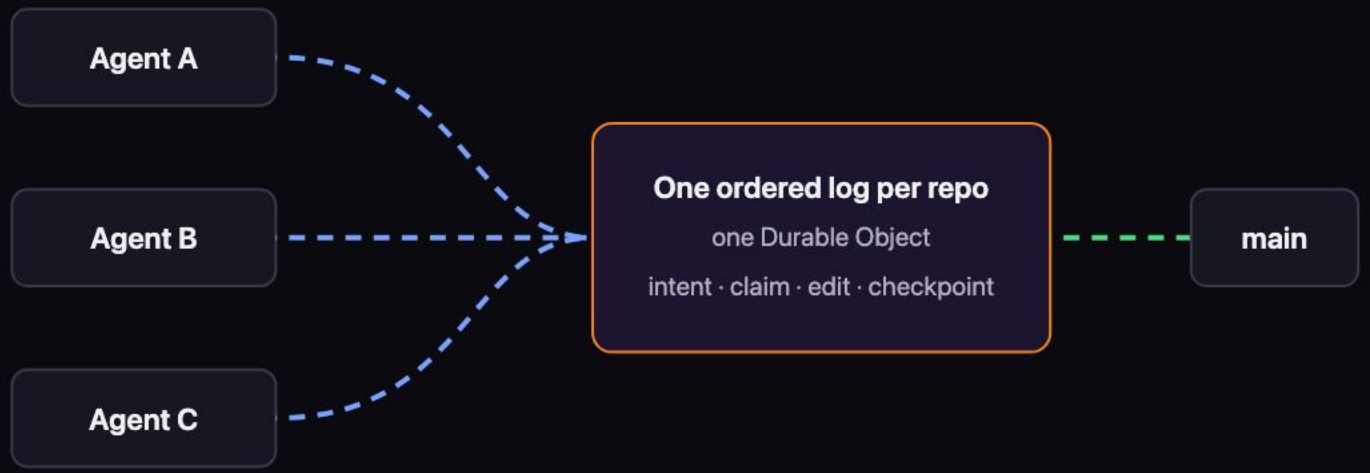
Claude Code | Codex | OpenCode | Hermes | Gemini CLI | Cursor

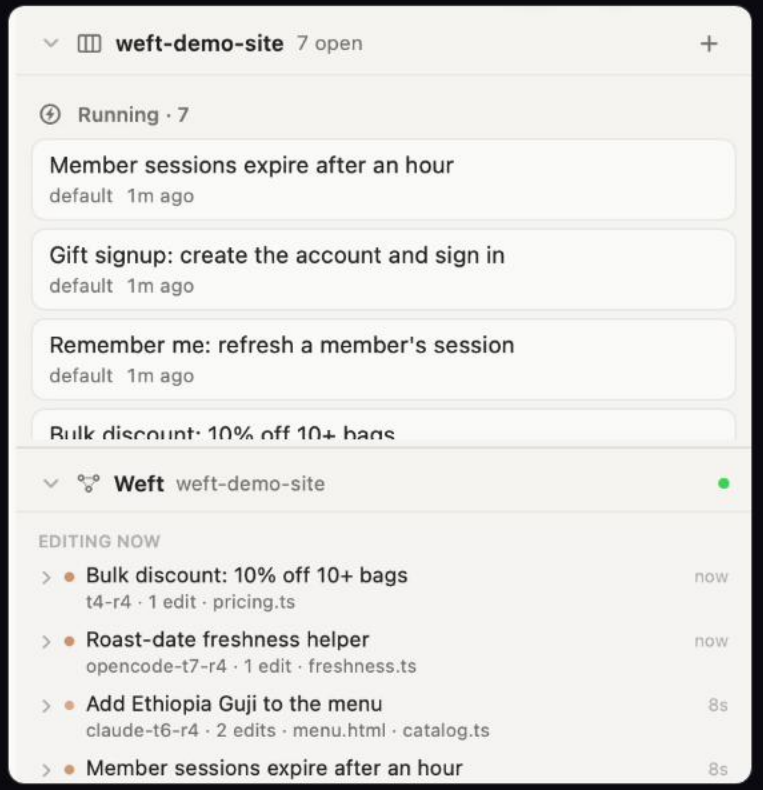
File watcher: any editor, people too

○ live today ○ adapter written ◌ observes, can't refuse

The protocol runs on a laptop, no Cloudflare account. **Cloudflare is how it scales.**

How do agents know what other agents are working on?





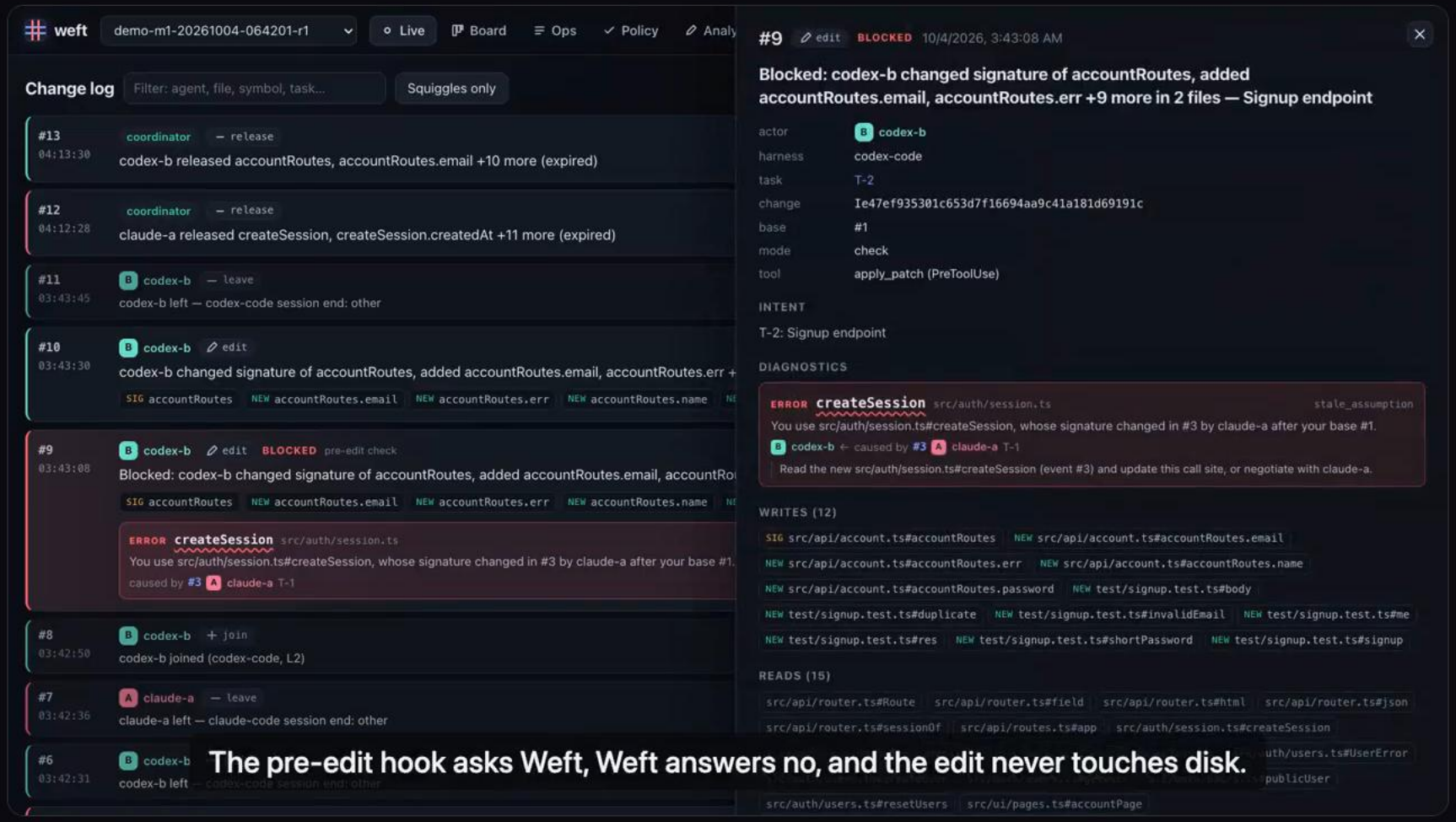
The Otto run: 7 cards, who is editing what, live

Every edit is checked against the **functions and types other agents just changed**.

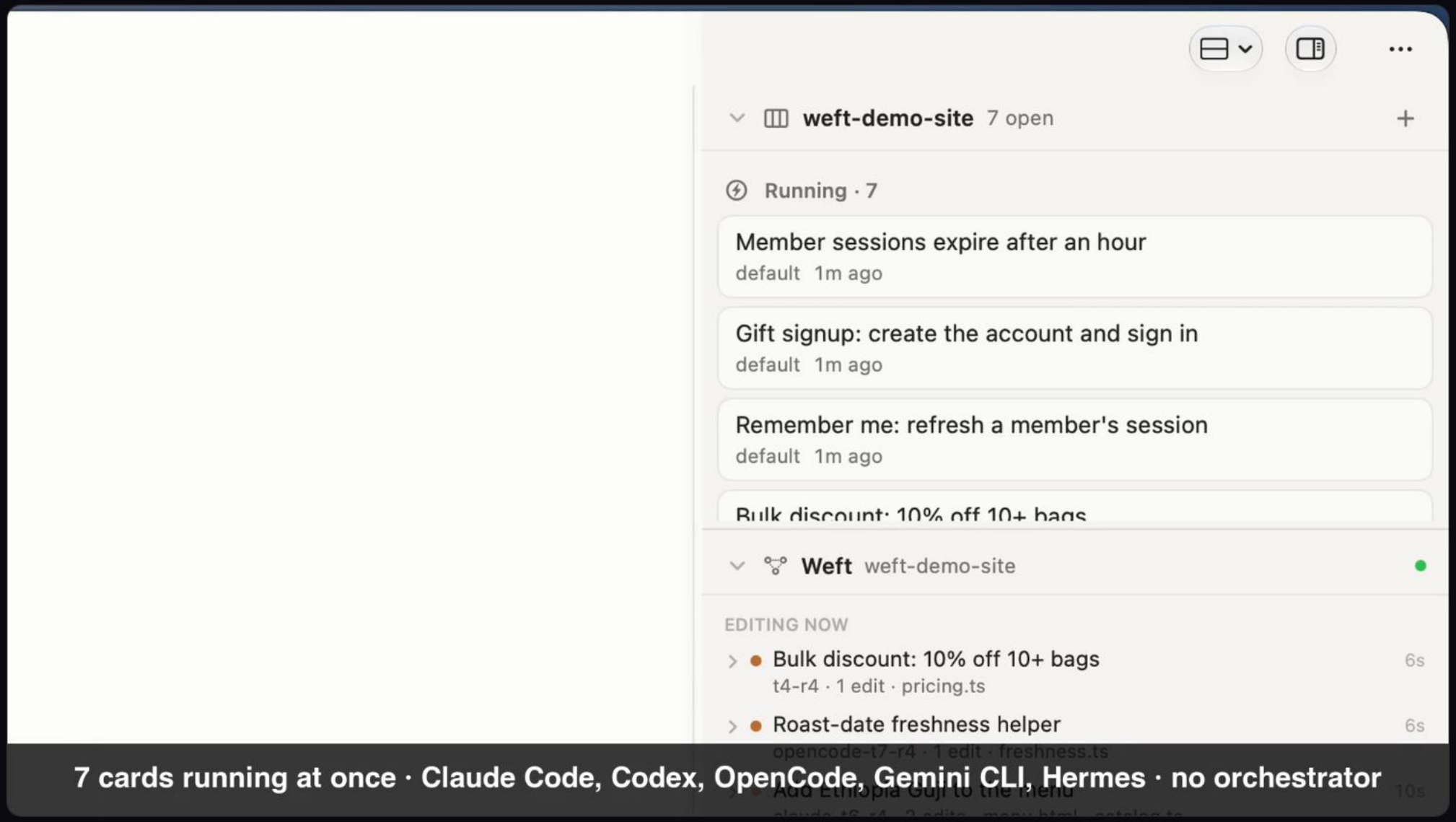
3 · QUESTION 2 OF 4 · CONFLICTING CHANGES · WITHOUT WEFT, A REAL RUN

```
 3 files changed, 28 insertions(+), 8 deletions(-)
main $ git merge --no-edit agent-b  # POST /api/signup
Merge made by the 'ort' strategy.
 src/api/account.ts  | 24 +++++++++++++++--
 test/signup.test.ts | 96 ++++++++++++++
```

3 · QUESTION 2 OF 4 · CONFLICTING CHANGES · WITH WEFT



Claude Code changes `createSession` → Codex's edit is refused → it adapts → accepted



7 cards running at once · Claude Code, Codex, OpenCode, Gemini CLI, Hermes · no orchestrator

7 cards at once · candidates on 5 harnesses · **no agent in charge** · 6 squiggles, all rerouted · all 7 landed

What happens when they make conflicting changes?

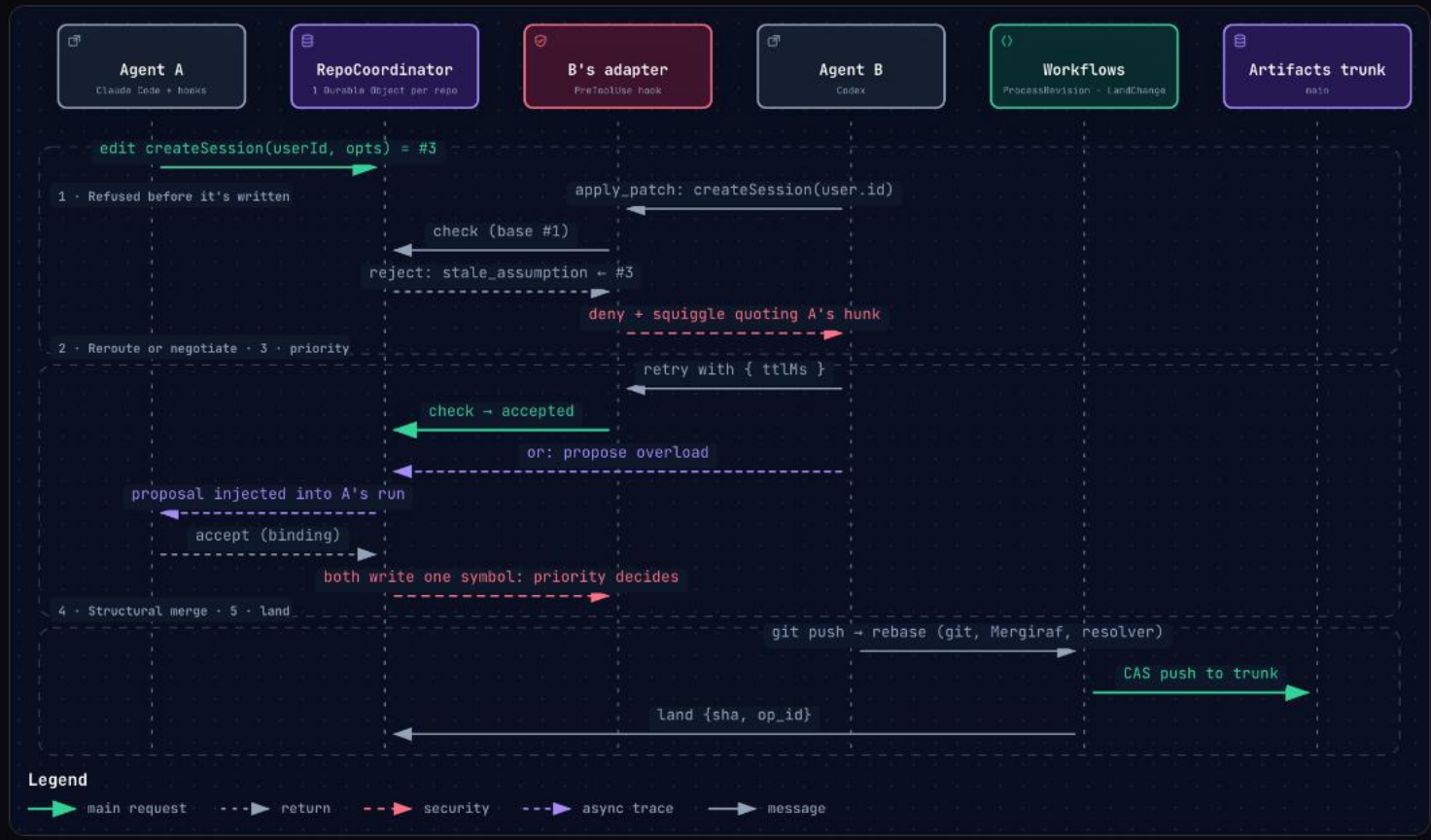
1 Refused before it's written
a stale_assumption squiggle quotes the other agent's change

2 Reroute or negotiate
propose an overload · accept · counter

3 Priority decides who adapts
same rule every time: no deadlocks

4 Structural merge
Mergiraf, not line-by-line

5 Land
compare-and-swap on trunk: main moves only if nothing moved



Traced from the code · deny → reroute as in the M1 run (Claude Code vs Codex) · interactive ↗

3 · QUESTION 3 OF 4

How do you review everything they produce?

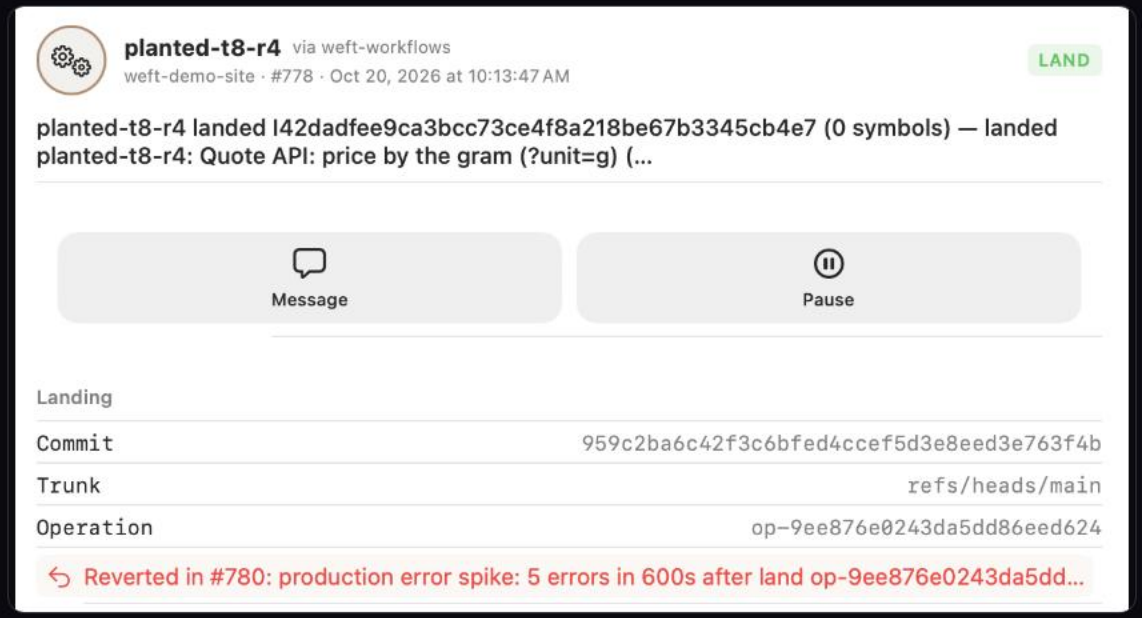
An evidence pipeline for every candidate:

Tests	Preview	Screenshots + pixel diff	Risk tier	Review agent acceptance criteria, fails closed

Low risk lands on its own. **Higher risk waits for a human.**

Production errors spike after a landing → Weft reverts it.

In the Otto run: a planted bug was reverted **32 s** after it landed.



3 · QUESTION 4 OF 4

How do you keep track of why?

```
Change-Id: I2e59ce5c1eaa4d57a28a2a5c8f9458bdcd8fd4f3
Task-Id:   agent-issue-4-continue
Agent-Id:  wt-4-agent-issue-4-continue
```

Every commit carries its change, task and agent.

Every squiggle cites the event that caused it (#834 ← #813).

The log replays byte for byte · each run's model transcript is stored.

A revert names the operation it undoes, and why.

WHY ARTIFACTS + CLOUDFLARE

Weft doesn't need Artifacts to work. **Artifacts is how it scales.**

Artifacts	a fork + a scoped token per agent, in one call · millions of repos
Queues	every push arrives as an event
Atomic landing	a lease push: trunk moves only if nothing moved
Durable Objects	exactly one authoritative log per repo

Red squiggles for teams of coding agents.

Coordination, not Orchestration

weft.elier.ai

John Nelson · johnaaronnelson.com · github.com/celador